

Planejamento de implementação - Focus Up

Objetivo: alinhar o projeto React + Firebase às capacidades do cronograma de referência (segurança, perfis, fluxos com estado, administração, qualidade de entrega e API de análise).

Etapa 1 - Fundação e segurança

- Variáveis de ambiente VITE_FIREBASE_* e VITE_ADMIN_EMAIL com .env.example documentado.
- Configuração Firebase validada em build de produção (src/config/env.js).
- Regras Firestore (firestore.rules) com leitura/escrita por utilizador, admin por e-mail, coleções solicitacoes, doacoes, analytics_daily.
- Índices compostos para consultas por userId + createdAt e filtros de doações (firestore.indexes.json).
- Registo de logins diários para métricas reais no painel admin.

Critérios de conclusão: npm run build com .env preenchido; firebase deploy --only firestore sem erros; login incrementa analytics_daily.

Etapa 2 - Modelo de dados e perfis

- Campo perfil no registo: aluno, mentor, apoiador (equivalente a tipos de utilizador do cronograma).
- Documento users com moedas, nivel_pet, gamificação existente.
- Coleções novas: solicitacoes, doacoes com historico de alterações de estado.

Critérios: novo utilizador guarda perfil; admin vê pedidos e contribuições associados ao userId.

Etapa 3 - Fluxo de solicitações

- Utilizador: criar e listar solicitações com estados pendente, aprovada, rejeitada.
- Admin: separador no painel para alterar estado e opcionalmente registar nota.

Critérios: transição de estado persiste no Firestore e aparece no histórico.

Etapa 4 - Fluxo de contribuições

- Utilizador: registar contribuição tipo tempo ou financeira (descrição + detalhe).
- Admin: filtro por tipo e alteração de estado pendente, aceite, recusada.

Critérios: filtros funcionam com índices deployados; utilizador só lê os seus registos.

Etapa 5 - Conta e painel admin

- Ecrã **Definições:** atualizar nome no Firestore; alterar palavra-passe (conta e-mail/senha) com reautenticação; mensagem informativa para login Google.
- Painel admin: gráfico de acessos com dados reais (últimos 7 dias); separadores Visão geral / Solicitações / Contribuições.

Critérios: métricas deixam de usar dados fictícios para a série de logins.

Etapa 6 - UX e modularização

- NotifyProvider + Snackbar para feedback consistente (substituição progressiva de alert).
- Serviços em src/services/ para analytics, solicitações e doações.
- Pomodoro com durações padrão 25 min / pausa 5 min.

Critérios: fluxos principais notificam sucesso/erro sem alert nativo nas áreas alteradas.

Etapa 7 - Qualidade e entrega

- docs/arquitetura.md com fluxos e coleções.
- docs/manual-checklist.md para testes antes de entrega.
- README com setup, deploy de regras, índices e API opcional.

Critérios: checklist percorrido sem bloqueios críticos.

Etapa 8 - API de análise

- Serviço Python FastAPI em /api: GET /health, POST /predict/score, POST /cluster/users.
- Documentação api/README_API.md e api/tests_api.http.

Critérios: uvicorn main:app responde e exemplos do README executam.

Dependências entre etapas

Fluxo sugerido: Etapa 1 -> Etapa 2 -> (Etapa 3 e Etapa 4 em paralelo) -> Etapa 5 -> Etapa 6 -> Etapa 7. A Etapa 8 (API) pode correr em paralelo desde a Etapa 1.

Riscos e mitigação

- Índice Firestore em falta: seguir o link do erro na consola e correr firebase deploy das regras e índices.
 - E-mail admin diferente do .env: ajustar também a função adminEmail em firestore.rules antes do deploy.
 - API Python opcional: entregar README e ficheiro tests_api.http mesmo sem hospedar a API.
-

Responsáveis (sugestão equipa)

- Etapas 1 a 2: backend e Firebase.
 - Etapas 3 a 4: fluxos full-stack.
 - Etapas 5 a 6: frontend e UX.
 - Etapa 7: QA e documentação.
 - Etapa 8: dados e ML leve.
-

Documento gerado para acompanhamento do projeto Focus Up; versão PDF produzida pelo script scripts/generate_planning_pdf.py.